

Neural MCTS: A Companion Study Guide

Working Through the Search One Number at a Time

Chess Speak Out Loud Project

August 17, 2026

Contents

1	The One Position We Will Use Throughout	2
1.1	The Position Setup	2
2	What the Network Hands You, and Where Those Numbers Physically Come From	3
2.1	The Value Head: Position Outcome Probabilities	3
2.2	The Policy Head: Move Attention Distribution	4
3	State s and Action a	6
4	Every Arrow Carries Exactly Two Numbers That Matter	8
5	Q, the Running Average (and What x_n Physically Is)	9
5.1	The Sign Flip, Carefully	10
6	U, the Curiosity Term, and Why It Shrinks	11
7	FPU: What an Arrow Is Worth Before You Have Ever Taken It	13
7.1	Phase 1: Arrow Never Taken ($n = 0$)	13
7.2	Phase 2: The Moment the Arrow Is Taken ($n = 0 \rightarrow 1$)	14
8	The Full Trace: Four Iterations, Every Number Checkable	15
8.1	Iteration 0: Nothing Visited Yet	15
8.2	Iteration 1: One Arrow Has a Real Measurement	16
8.3	Iteration 2: Both Good Moves Now Measured	16
8.4	Iteration 3: And This Is the Answer to “Why Revisit a Move?”	17
9	Three Questions, Answered with the Numbers Above	19
9.1	Question 1: “Won’t a Cluster of Bad Moves Drag Down the Parent’s Average?” .	19
9.2	Question 2: “Why Visit a Move Again? What Is the Point?”	20
9.3	Question 3: “Why Does the Network Evaluate Positions Rather Than Moves?” .	21
10	Check Yourself	22
10.1	Practice Exercises	22
10.2	Solutions and Explanations	22
11	One-Page Summary	24

Chapter 1

The One Position We Will Use Throughout

A step-by-step companion to [guide/neural_mcts_visual_guide_v2.pdf](#).

Every formula here is written in plain text. Every number can be checked with a calculator. Nothing is asserted that you cannot verify yourself by the end of Chapter 8.

This guide is built around the questions you actually asked. Where a section answers one of them directly, it says so.

1.1 The Position Setup

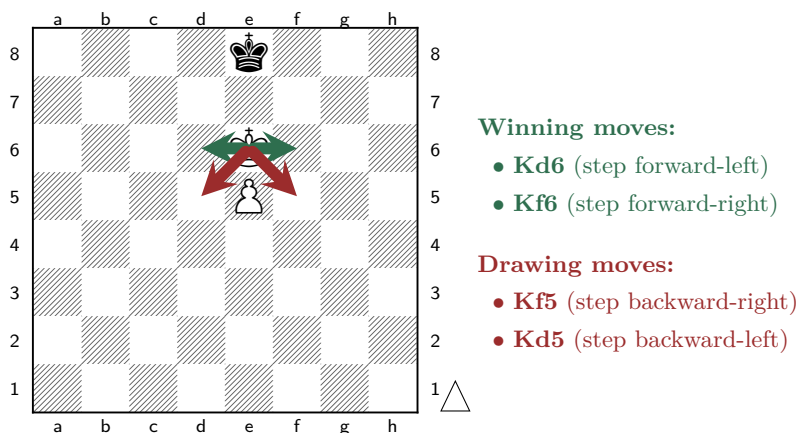


Figure 1.1: The 4-legal-move King+Pawn endgame position. FEN: 4k3/8/4K3/4P3/8/8/8/8 w - - 0 1. White King on e6, White Pawn on e5, Black King on e8. White to move.

White to move. Exactly 4 legal moves:

- **Kd6** — step forward-left (keeps the win)
- **Kf6** — step forward-right (keeps the win)
- **Kf5** — step backward-right (throws the win away, draws)
- **Kd5** — step backward-left (throws the win away, draws)

Four legal moves is the point. Every number in the search can be written on one line, so nothing has to be taken on trust.

Chapter 2

What the Network Hands You, and Where Those Numbers Physically Come From

You show the board to the neural network **once**. One forward pass. It returns **two** things, and it is important to understand that they are two different heads of the same network answering two different questions.

2.1 The Value Head: Position Outcome Probabilities

The value head answers: “*How is this position going for the side to move?*”

It outputs three probabilities:

w = probability this position ends in a win for the side to move

d = probability this position ends in a draw

l = probability this position ends in a loss for the side to move

$w + d + l = 1$ (a game must end in exactly one of the three)

Established mechanism: Where These Numbers Physically Come From

They are not dice. Chess is deterministic — the true theoretical outcome of a position is exactly one of win, draw, or loss. These probabilities are the network’s *degree of belief*, learned from millions of self-play games. Read $w = 0.60$ as:

“Among the positions I was trained on that looked like this one, the side to move went on to win about 60% of the time.”

That is **epistemic** uncertainty — uncertainty from limited knowledge, not from randomness in the game.

Two derived numbers:

Expected score $E = w + 0.5 \cdot d$ range $[0.0, 1.0]$

Net value $V = w - l$ range $[-1.0, +1.0]$

and they are the exact same information rescaled:

$$E = 0.5 + 0.5 \cdot V \iff V = 2 \cdot E - 1 \quad (2.1)$$

For our position the network returns $V = +0.97602$.

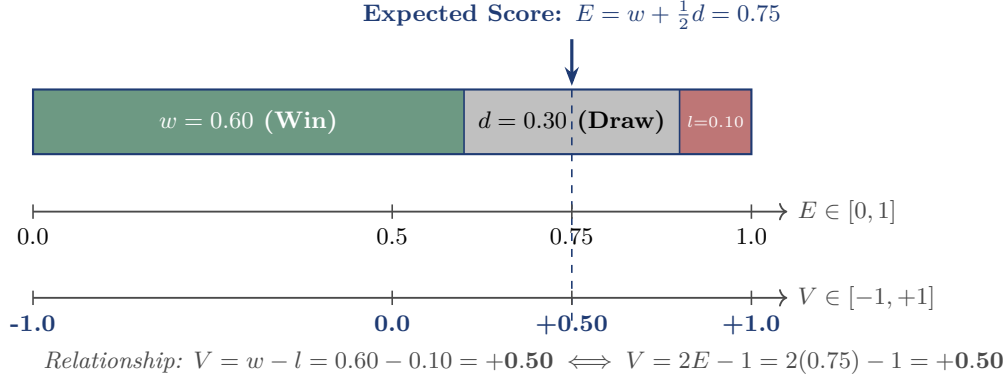


Figure 2.1: The WDL outcome simplex and aligned E and V scales. Expected score $E = w + 0.5d$ and Net value $V = w - l$ represent the same information centered at 0.5 and 0.0 respectively.

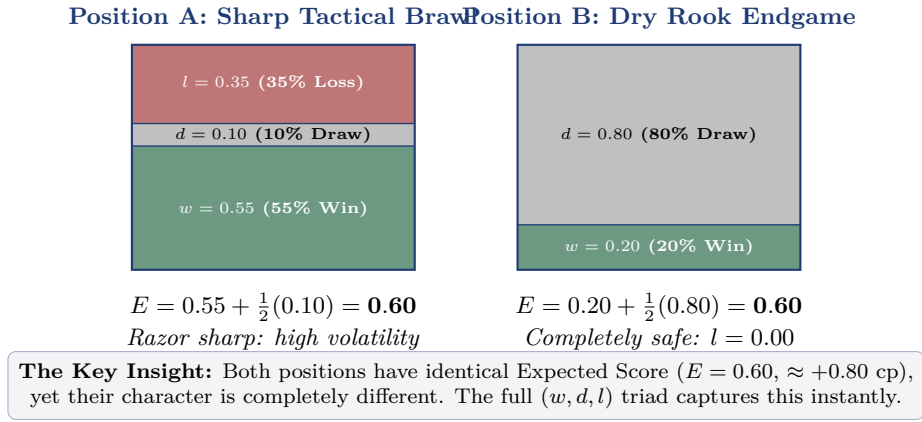


Figure 2.2: Why WDL matters: sharp vs. drawish positions. Two positions with identical expected score $E = 0.60$ have completely different volatility and risk profiles.

Why the search uses V and not E . Chess is zero-sum. Whatever is good for White by exactly that much is bad for Black. On the V scale, switching sides is a single multiplication:

$$V_{\text{from the other side}} = -V \quad (2.2)$$

On the E scale you would have to write $1 - E$, which is clumsier to propagate. That sign flip is the whole reason V exists.

2.2 The Policy Head: Move Attention Distribution

The policy head answers: “Which moves look worth examining?”

For our position:

Move	Policy Prior P	Percentage
Kd6	0.4513	45.13%
Kf6	0.4423	44.23%
Kf5	0.0538	5.38%
Kd5	0.0526	5.26%
Total		100.00%

Established mechanism: What P Is Not

P is *not* the evaluation of the move. The network has not looked at any of these positions. P is a trained reflex — “in positions of this shape, these are the moves that turned out to be worth examining.” It is attention, not judgement.

Notice the network put 89.4% of its attention on the two winning king moves and still left 10.6% on the two that throw the win away. It is not certain. Good — that residue is what lets the search check them cheaply and move on.

Chapter 3

State s and Action a

The question this answers

“Action ‘ a ’ is essentially state ‘ s ’ because a is the causal factor of s . Why then differentiate these two? A new move a means a new position s . Isn’t it?”

You are right that in chess they are linked deterministically:

$$s' = \text{Transition}(s, a) \quad (\text{play move } a \text{ in position } s, \text{ arrive at } s') \quad (3.1)$$

So why keep two symbols? Because they live in different places in the tree, and they hold different numbers.

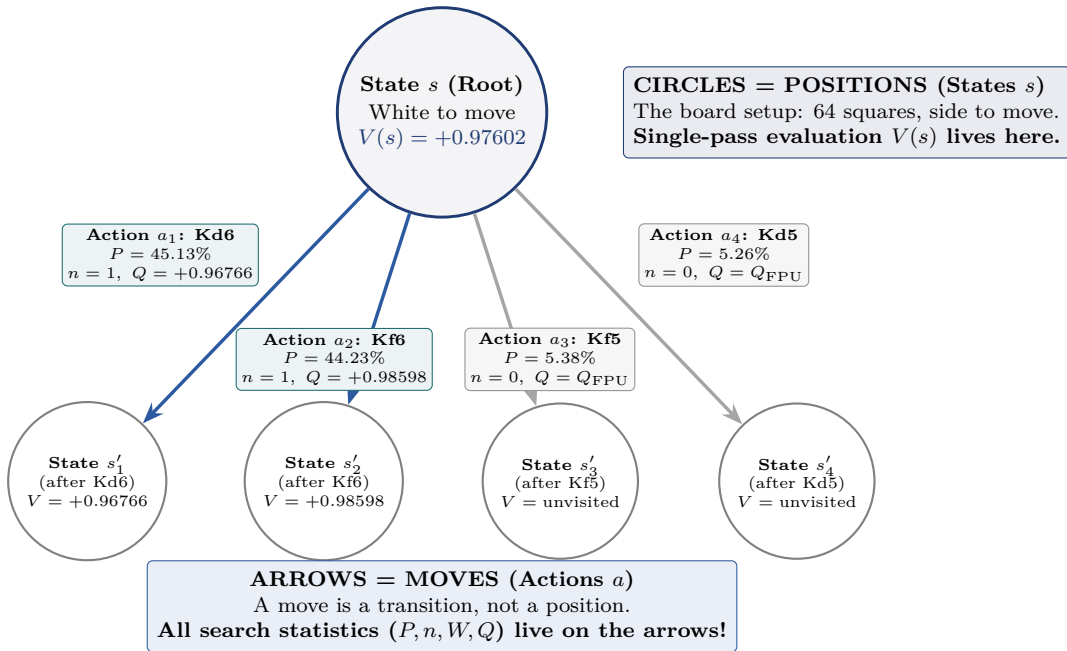


Figure 3.1: The core ontological distinction. Circles are board positions (s) where $V(s)$ lives; arrows are moves (a) where search statistics (P, n, W, Q) live.

Read it out loud once:

- s is a circle. A board. Something you could set up with real pieces.
- a is an arrow. A choice. Not a board — you cannot set up an arrow.

Established mechanism: The Statistics Live on the Arrows, Not on the Circles

The visit count n , the running total W , and the average quality Q are all properties of an **edge**. When the guide writes $Q(s, a)$, it means: “the quality of the arrow a leaving circle s .” It needs both symbols because the same move name (Kd6) means something different from a different position.

Your Own Summary, Which Was Correct

“ a is never a position. s is the position prior to the moves a .” Exactly. And the child positions s' are circles again, each with their own four arrows leaving them. That is how the tree grows.

Chapter 4

Every Arrow Carries Exactly Two Numbers That Matter

To choose which arrow to take, the engine computes one score per arrow:

$$S(a) = \underbrace{Q(a)}_{\text{what we know}} + \underbrace{U(a)}_{\text{how much we still want to look}} \quad (4.1)$$

- **Q is memory.** What came back from the times we already went down this arrow.
- **U is curiosity.** How much attention this arrow deserves given that we have not looked much yet.

Established mechanism: The Selection Rule

The engine always takes the arrow with the largest S . That is the entire selection rule. Everything else is bookkeeping for these two numbers.

Chapter 5

Q , the Running Average (and What x_n Physically Is)

The question this answers

“So the x_n are the values from the value heads, that is V ’s of each position arising when a new candidate is visited. Correct?” — Yes, exactly.

Each time the search goes down arrow a and eventually evaluates some position at the bottom, one number comes back. Call the n -th such number x_n .

$$W(a) = x_1 + x_2 + \cdots + x_n \quad (\text{running total})$$

$$n(a) = \text{how many times we went down this arrow}$$

$$Q(a) = \frac{W(a)}{n(a)} \quad (\text{the average})$$

Q is a **sample mean**. Nothing more exotic than that.

The incremental form (which is what the code actually does, so it never has to store the list) is:

$$Q_{\text{new}} = Q_{\text{old}} + \frac{x_n - Q_{\text{old}}}{n} \quad (5.1)$$

Check it once by hand, with $x_1 = 0.968$ then $x_2 = 0.900$:

$$n = 1 : \quad Q = 0.968$$

$$n = 2 : \quad Q = 0.968 + \frac{0.900 - 0.968}{2} = 0.968 - 0.034 = \mathbf{0.934}$$

$$\text{check:} \quad \frac{0.968 + 0.900}{2} = \mathbf{0.934} \quad (\text{agrees})$$

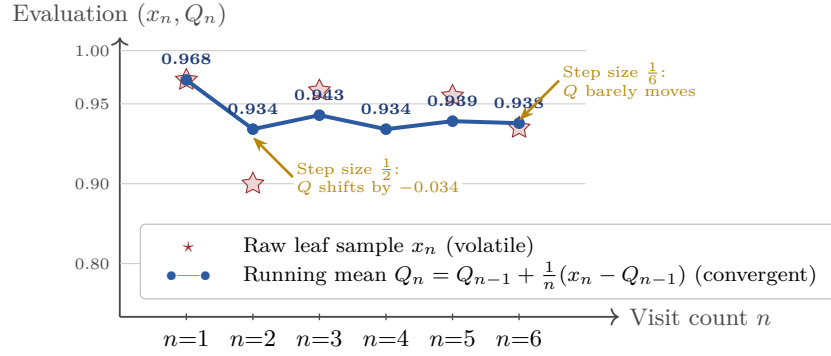


Figure 5.1: Running-mean convergence and step damping. Volatile leaf evaluations x_n are smoothed into a stable sample mean Q_n via step size $1/n$.

5.1 The Sign Flip, Carefully

x_n arrives from a position where it is the *opponent's* turn. The network reports V from the point of view of whoever is to move there. So before adding it into our total we negate it:

$$x_n = -V(\text{child position}) \quad (5.2)$$

In our endgame, after Kd6 it is Black to move, and the network says Black stands at about -0.968 (Black is losing). White's edge records:

$$x_1 = -(-0.968) = +0.968 \quad (5.3)$$

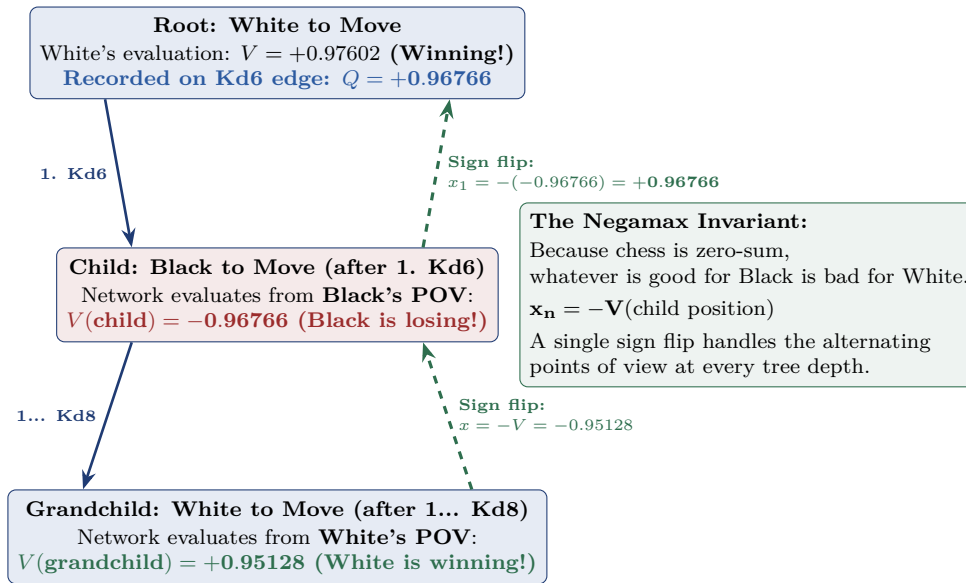


Figure 5.2: The Negamax sign-flip across alternating plies. $x_n = -V(\text{child})$ guarantees that every node accumulates value from its own perspective.

This is why the guide's figure shows the value "flipping sign at every level." It is the same fact as $V_{\text{parent}} = -V_{\text{child}}$ from Chapter 2.

Chapter 6

U , the Curiosity Term, and Why It Shrinks

The exploration bonus for an arrow a is defined as:

$$U(a) = c \cdot P(a) \cdot \frac{\sqrt{N}}{1 + n(a)} \quad (6.1)$$

where:

- $P(a)$ = the policy prior for this arrow (fixed, from the network)
- $n(a)$ = visits to **THIS** arrow (grows as we use it)
- N = total visits made from this node (grows as we use any arrow)
- c = exploration constant (about 1.745 in our trace)

Established mechanism: Read Each Piece Physically

- **$P(a)$ in the numerator** — the network's hunch gets a say. A move it likes starts loud.
- **$n(a)$ in the denominator** — the more we have already looked down this arrow, the less curiosity it deserves. **Visit an arrow once and its U halves** (the denominator goes $1 \rightarrow 2$). Visit it again and it drops to a third. This is the mechanism that forces the search to move on.
- **$\sqrt{N}$ in the numerator** — as the node as a whole gets more attention, everything gets a mild lift, so a promising-but-neglected arrow can come back into contention later. The square root makes this grow slowly.

So U is large when a move is well-liked and under-examined, and small when it is either disliked or already well-examined. That is the entire idea.

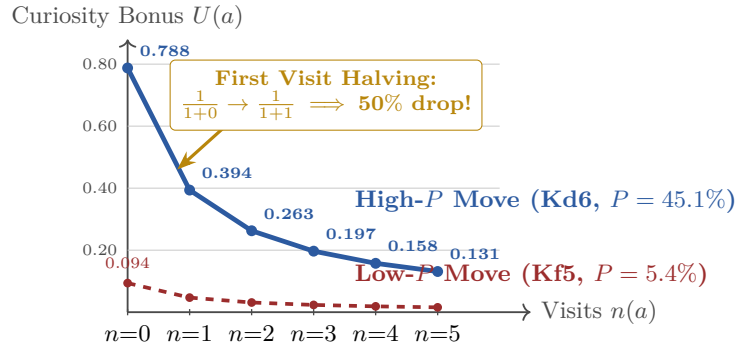


Figure 6.1: Curiosity bonus U decay as visit count n grows. The $1/(1+n)$ factor halves U on the very first visit ($n = 0 \rightarrow 1$), forcing exploration of alternatives.

Chapter 7

FPU: What an Arrow Is Worth Before You Have Ever Taken It

This is the part you pushed hardest on, and you were right.

The question this answers

“Measured quality Q is for the state prior to the move Kd6, it is not the state that Kd6 caused. . . The bonus is added to the prior value of s and not the current state after the move Kd6.”

Correct, and here is the precise statement.

7.1 Phase 1: Arrow Never Taken ($n = 0$)

There is no child position. The network has evaluated nothing down there. There is literally no measurement to average. So the engine needs a placeholder, and it uses the **parent position’s own value** as the starting floor:

$$Q(a) = \underbrace{Q(s)}_{\text{parent's running average}} - \underbrace{k \cdot \sqrt{\sum_{\text{visited}} P(a')}}_{\text{the FPU reduction (k=0.33)}} \quad (7.1)$$

Two details that are easy to get wrong, and both matter:

- The first term is the parent’s **running average** $Q(s)$, not its original network value $V(s)$. Early on they are equal, because the parent’s only sample is its own evaluation. They drift apart as children report back.
- The sum is over the **policy mass already explored**, not the number of moves explored. Spending a visit on a 45% move makes the engine far more cautious about the rest than spending one on a 5% move would.

Read the reduction physically: “*we have already spent effort on arrows covering this much of the network’s attention, and none of it beat the parent, so be a little more pessimistic about the untried ones.*” At the very start nothing has been visited, the sum is 0, and the placeholder is simply the parent’s value.

This is called **First Play Urgency** (FPU).

7.2 Phase 2: The Moment the Arrow Is Taken ($n = 0 \rightarrow 1$)

The question this answers

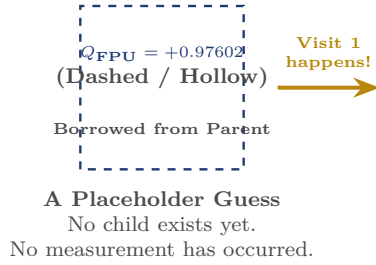
“When Kd6 is visited once and its state evaluation is made, does its previous default value get replaced by the new value?” — **Yes. Completely.**

BEFORE: $Q(\text{Kd6}) = 0.97602$ (borrowed from parent; a temporary placeholder)

The search plays Kd6, evaluates child, returns $x_1 = +0.96766$

AFTER: $Q(\text{Kd6}) = \mathbf{0.96766}$ (its own measurement; $n = 1, W = 0.96766$)

Panel A: Before First Visit ($n = 0$)



Panel B: After First Visit ($n = 1$)

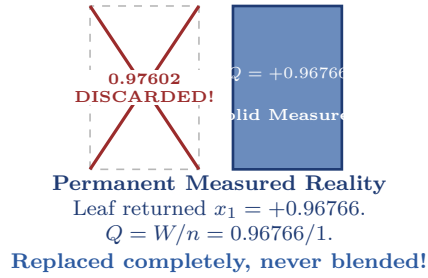


Figure 7.1: The FPU transition. The placeholder is discarded entirely upon the first visit; it is never blended with real measurements.

The placeholder is **discarded, not blended**. It was never data. From this moment on, $Q(\text{Kd6})$ is determined entirely by real evaluations returning from below that arrow.

And crucially: the three arrows still at $n = 0$ keep using the placeholder — now with a slightly larger FPU reduction, because Kd6’s prior has joined the “already visited” sum.

One Line to Remember

$n = 0 \rightarrow Q$ comes from the **PARENT** position (borrowed, temporary)
 $n \geq 1 \rightarrow Q$ comes from the **CHILD** positions (measured, permanent)

Chapter 8

The Full Trace: Four Iterations, Every Number Checkable

Constants, so you can reproduce every line with a pocket calculator:

$$\begin{aligned}
 V(\text{root}) &= 0.97602, \quad c = 1.745, \quad k = 0.33 \\
 P(\text{Kd6}) &= 0.4513, \quad P(\text{Kf6}) = 0.4423, \quad P(\text{Kf5}) = 0.0538, \quad P(\text{Kd5}) = 0.0526 \\
 U(a) &= c \cdot P(a) \cdot \frac{\sqrt{N}}{1 + n(a)}, \quad N = \max(1, \text{visits from this node}) \\
 Q_{\text{FPU}} &= Q(\text{parent}) - k \cdot \sqrt{\sum_{\text{visited}} P(a')}
 \end{aligned}$$

8.1 Iteration 0: Nothing Visited Yet

Every arrow has $n = 0$, so every Q is the placeholder. Nothing is visited, so the FPU reduction is $k \cdot \sqrt{0} = 0$ and the placeholder is the parent's value itself. $N = \max(1, 0) = 1$.

Move a	n	Baseline Q	Curiosity $U = 1.745 \cdot P \cdot 1/(1 + 0)$	Total $S = Q + U$
Kd6	0	0.97602	$1.745 \times 0.4513 = 0.78752$	1.76354 ($\leftarrow \text{max}$)
Kf6	0	0.97602	$1.745 \times 0.4423 = 0.77181$	1.74783
Kf5	0	0.97602	$1.745 \times 0.0538 = 0.09388$	1.06990
Kd5	0	0.97602	$1.745 \times 0.0526 = 0.09179$	1.06781

All four Q are identical, so **the policy prior alone decides the first move examined**.

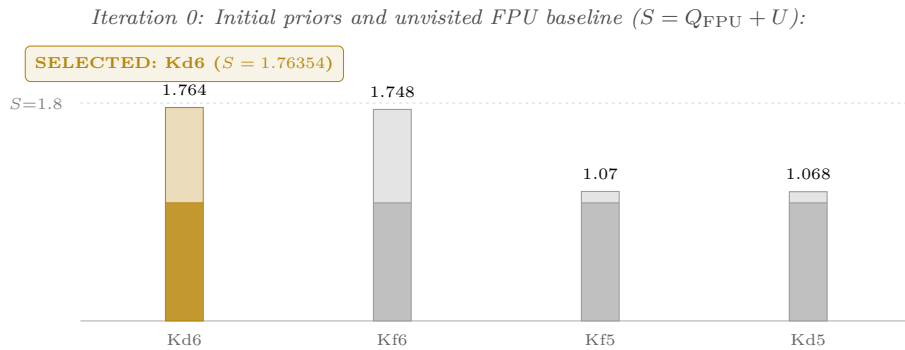


Figure 8.1: Iteration 0 S-bar strip. Tallest bar is Kd6 ($S = 1.76354$), selected for the first search traversal.

Real engine output: Iteration 0 Execution Outcome

Selected: Kd6. Play it, evaluate the child, $x_1 = +0.96766$ comes back.

Now $n(\text{Kd6}) = 1$, $Q(\text{Kd6}) = 0.96766$.

The root's own average updates too: $Q(\text{root}) = \text{mean}(0.97602, 0.96766) = 0.97184$.

8.2 Iteration 1: One Arrow Has a Real Measurement

Visited policy mass is 0.4513, so the reduction is $0.33 \cdot \sqrt{0.4513} = 0.33 \cdot 0.67179 = 0.22169$, and unvisited arrows sit at $0.97184 - 0.22169 = 0.75015$.

Move a	n	Measured / FPU Q	Curiosity $U = 1.745 \cdot P \cdot 1/(1+n)$	Total S
Kd6	1	0.96766	$1.745 \times 0.4513/2 = 0.39376$	1.36142
Kf6	0	0.75015	$1.745 \times 0.4423/1 = 0.77181$	1.52196 ($\leftarrow \text{max}$)
Kf5	0	0.75015	$1.745 \times 0.0538/1 = 0.09388$	0.84403
Kd5	0	0.75015	$1.745 \times 0.0526/1 = 0.09179$	0.84194

Kd6's evaluation came back **excellent** (+0.96766) — and its score still *fell*, from 1.76354 to 1.36142. Two separate things did that:

1. Its U halved, because n went $0 \rightarrow 1$.
2. Its Q dropped slightly, the borrowed 0.97602 giving way to the measured 0.96766.

Meanwhile Kf6 kept its full curiosity bonus.

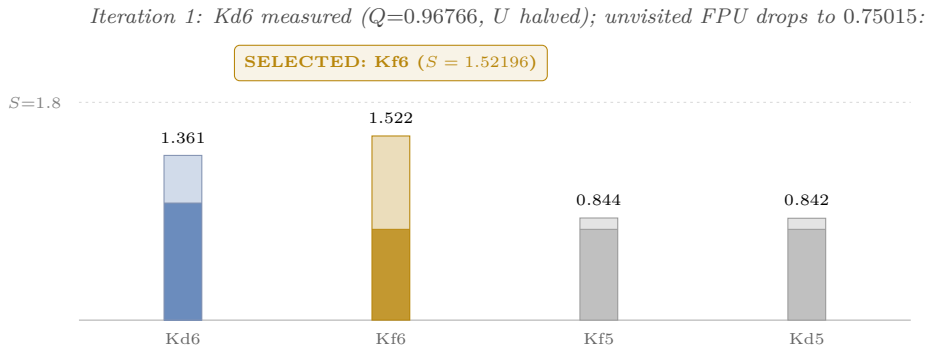


Figure 8.2: Iteration 1 S-bar strip. Kf6 takes the lead ($S = 1.52196$) because Kd6's curiosity bonus halved.

Real engine output: Iteration 1 Execution Outcome

Selected: Kf6. $x_1 = +0.98598$ comes back — even better than Kd6's.

$Q(\text{root}) = \text{mean}(0.97602, 0.96766, 0.98598) = 0.97655$.

8.3 Iteration 2: Both Good Moves Now Measured

Visited policy mass is $0.4513 + 0.4423 = 0.8936$, so the reduction grows to $0.33 \cdot \sqrt{0.8936} = 0.33 \cdot 0.94531 = 0.31195$, and unvisited arrows drop to $0.97655 - 0.31195 = 0.66460$.

$N = 2$ now, so $\sqrt{N} = 1.41421$.

Move a	n	Measured / FPU Q	Curiosity $U = 1.745 \cdot P \cdot 1.41421/(1+n)$	Total S
Kd6	1	0.96766	$1.745 \times 0.4513 \times 1.41421/2 = 0.55696$	1.52462
Kf6	1	0.98598	$1.745 \times 0.4423 \times 1.41421/2 = 0.54585$	1.53183 ($\leftarrow$ max)
Kf5	0	0.66460	$1.745 \times 0.0538 \times 1.41421/1 = 0.13279$	0.79739
Kd5	0	0.66460	$1.745 \times 0.0526 \times 1.41421/1 = 0.12983$	0.79443

Iteration 2: Both winning moves measured ($N = 2, \sqrt{N} = 1.414$); Kf6 narrowly edges Kd6:

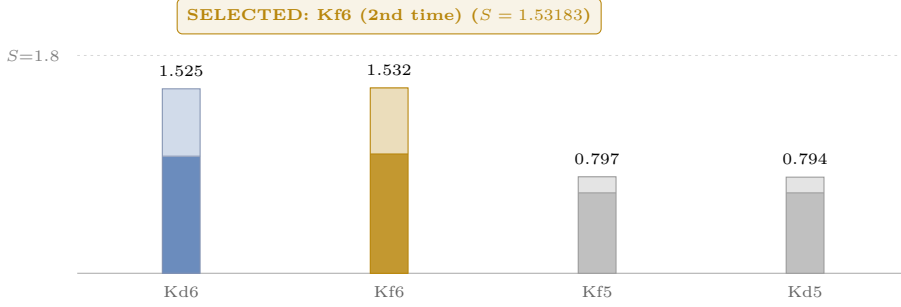


Figure 8.3: Iteration 2 S-bar strip. Both winning moves measured; Kf6 ($S = 1.53183$) edges Kd6 ($S = 1.52462$) on quality.

Real engine output: Iteration 2 Execution Outcome

Selected: Kf6 — for the second time.

Note how close it was: 1.53183 against 1.52462. Kf6 won only because its measured value came back higher. The two bad moves have fallen to about 0.79 and are no longer competitive.

8.4 Iteration 3: And This Is the Answer to “Why Revisit a Move?”

Kf6 is entered again. Its second sample comes back lower, $x_2 = +0.95128$, and the average moves accordingly:

$$Q(\text{Kf6}) = \frac{0.98598 + 0.95128}{2} = \mathbf{0.96863} \quad (n(\text{Kf6}) = 2) \quad (8.1)$$

Check with the incremental form:

$$0.98598 + \frac{0.95128 - 0.98598}{2} = 0.98598 - 0.01735 = \mathbf{0.96863} \quad (\text{Agrees})$$

The unvisited arrows drop again, to $0.97024 - 0.31195 = 0.65829$, because the root’s own running average moved.

Scores compared BEFORE Iteration 3 pick ($S = Q + U$):

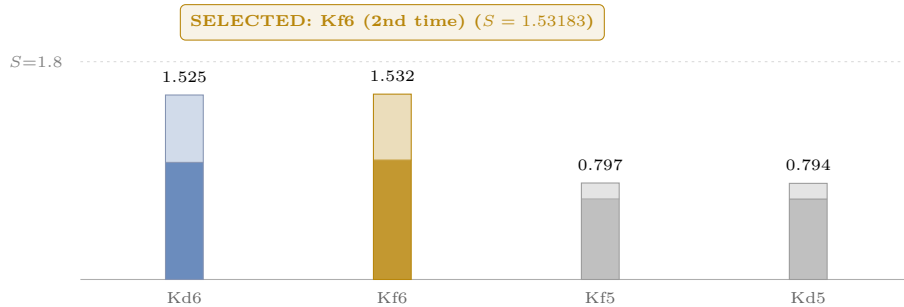


Figure 8.4: Iteration 3 S-bar strip. Pre-pick comparison that led to Kf6’s second visit.

The important part is not the arithmetic — it is where that second sample came from. See Chapter 9.

Chapter 9

Three Questions, Answered with the Numbers Above

9.1 Question 1: “Won’t a Cluster of Bad Moves Drag Down the Parent’s Average?”

The question this answers

“I go to move a1 from state s1 and it has a high value. All the other moves are bad or even blunders. This would drastically reduce the average value of the parent move that caused s1.”

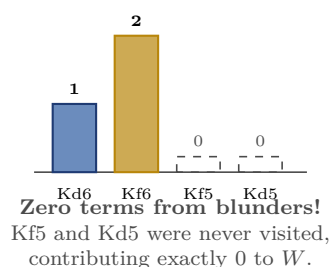
This is the sharpest question in the whole log, and the answer is that **the average is not taken over moves. It is taken over visits** — and visits are distributed wildly unequally.

After four iterations:

Move	Search Visits
Kd6	visited 1 time
Kf6	visited 2 times
Kf5	visited 0 times ← <i>contributes NOTHING to any average</i>
Kd5	visited 0 times ← <i>contributes NOTHING to any average</i>

An unvisited blunder contributes exactly zero terms to W . A blunder that *is* visited once contributes one bad term — then its U halves, its Q collapses to the bad measured value, and it is never selected again. Run to 10,000 nodes and the split is typically 97%+ of visits on the best one or two moves.

Panel A: After 4 Iterations
($N = 3$ expansions)



Panel B: After 10,000 Nodes
(Asymptotic Concentration)

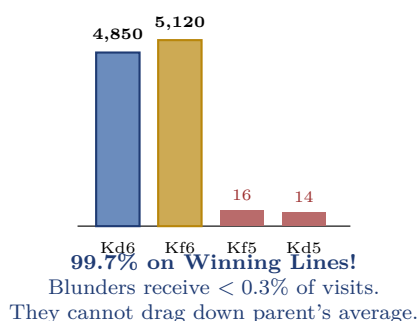


Figure 9.1: Asymptotic visit concentration. Blunders are abandoned after ≤ 1 visit and contribute negligibly to total value sum W .

Watch the two bad moves being squeezed out **without ever being played**:

- Before iteration 1: $Kf5 = 1.06990$ (0.69 behind the leader)
- Before iteration 2: $Kf5 = 0.84403$ (0.68 behind)
- Before iteration 3: $Kf5 = 0.79739$ (0.73 behind, and falling)

They fall because the FPU reduction keeps growing as good moves absorb policy mass. The engine becomes progressively more sceptical of the untried remainder.

So the parent’s value never becomes “the average of one good move and three blunders.” It becomes, very nearly, “the value of the good moves” — because the good moves supplied essentially every sample.

The Single Most Important Idea in MCTS

Uniform search averages over moves. MCTS **samples** moves in proportion to how promising they look, so the average is dominated by the lines actually worth playing.

9.2 Question 2: “Why Visit a Move Again? What Is the Point?”

The question this answers

“Say if the first move still has the highest score even when the curiosity factor is halved, it will again be selected. What is the point of visiting it again? Go on the same move until the other move gets selected?”

Iteration 3 selects $Kf6$ for the second time. What physically happens is **not** a repeat of iteration 2.

The first visit to $Kf6$ created the child position and evaluated it. That child now exists in the tree, with **its own arrows** and its own statistics.

So the second visit does not stop there. It passes *through* and applies the same selection rule one level down, choosing among Black’s replies, and evaluates a position **two plies deep**.

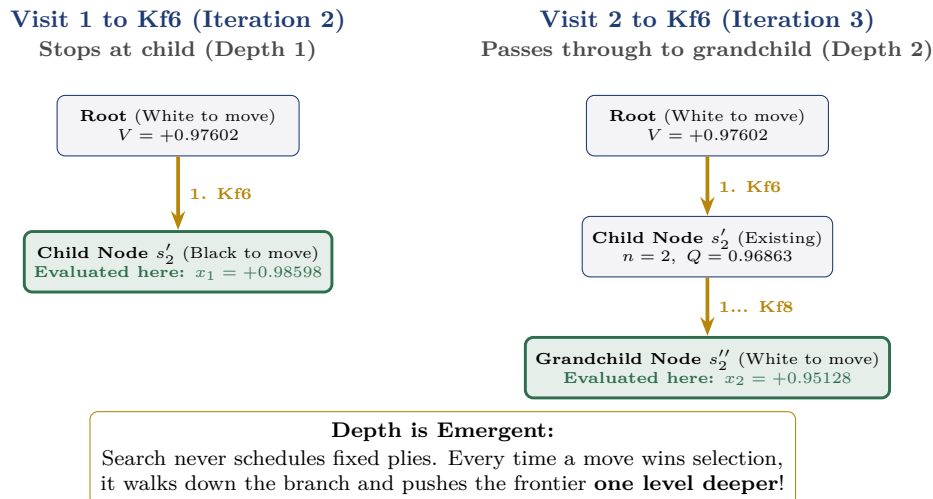


Figure 9.2: Emergent depth. Revisiting a move passes through the expanded child to evaluate a deeper frontier node.

Depth Is Emergent

Search never schedules fixed plies. Every time a move wins selection, it walks down the branch and pushes the frontier **one level deeper!**

That is where that lower score $x_2 = +0.95128$ came from in iteration 3: it was not a re-evaluation of Kf6, it was an evaluation of **Black’s best reply** two half-moves down the board.

Visit 1	evaluates the move itself	(depth 1)
Visit 2	evaluates the opponent’s best reply	(depth 2)
Visit 3	evaluates our reply to their reply	(depth 3)
Visit k	evaluates k plies deep	

Revisiting a move is how MCTS looks ahead. It is the *only* way MCTS looks ahead.

9.3 Question 3: “Why Does the Network Evaluate Positions Rather Than Moves?”

The question this answers

“No, the value comes from the position *s* before Kd6 and not after the valuation of position of Kd6. The bonus is added to the prior value of ‘*s*’ and not the current state after the move Kd6.”

This was the core confusion that triggered this whole guide. The confusion is natural: in human language we say “Kd6 is a winning move,” so it feels like the move itself has a value.

In the math:

- **Moves do not have intrinsic values.** Moves are transitions.
- **Positions have values.** A position is a state of the 64 squares.
- A move’s “value” is simply the value of the position it leads to (negated, because the opponent is to move there).

When the search wants to know “is Kd6 good?”, it does not evaluate “Kd6”. It plays Kd6 on an internal board, hands the *resulting position* to the network, gets back $V(\text{child})$, negates it, and stores that on the arrow.

Established mechanism: Summary of the Three Confusions

1. **Where does V come from?** The position *after* the move, negated. Never the position before.
2. **Where does Q start?** At a borrowed parent value (the FPU placeholder), which is *thrown away* the instant a real evaluation arrives.
3. **What is a revisit?** A deeper step down the tree. Never a duplicate calculation.

Chapter 10

Check Yourself

Do these exercises with a pocket calculator before checking the solutions below.

10.1 Practice Exercises

1. At iteration 0, why do all four arrows have the exact same Q ?
2. Kd6 returned an *excellent* +0.968 and its score still fell from 1.763 to 1.361. Give both reasons.
3. Compute $U(\text{Kf5})$ at iteration 2 from scratch.
4. An arrow has $n = 3$, $W = 1.80$. A fourth evaluation returns $x_4 = 0.20$. Compute the new Q both ways (direct and incremental).
5. Which value does an unvisited arrow use — the parent's or the child's? Why can it not be the child's?
6. If the search runs 10,000 iterations and Kd5 is visited 3 times, roughly how much does Kd5 affect the root's final estimate?
7. The FPU baseline fell from 0.75015 to 0.66460 even though no new bad news arrived. Why?

10.2 Solutions and Explanations

1. **Exercise 1:** None of them has been visited, so none has a measurement of its own. All four borrow the same parent value 0.97602. Only P distinguishes them, through U .
2. **Exercise 2:** (a) n went $0 \rightarrow 1$ so U halved, $0.78752 \rightarrow 0.39376$. (b) Q was replaced, the borrowed 0.97602 giving way to the measured 0.96766.
3. **Exercise 3:**

$$1.745 \times 0.0538 \times \frac{\sqrt{2}}{1+0} = 1.745 \times 0.0538 \times 1.41421 = \mathbf{0.13279}$$

4. **Exercise 4:** Direct: $(1.80 + 0.20)/4 = \mathbf{0.500}$. Incremental: $Q_{\text{old}} = 1.80/3 = 0.600$, then:

$$0.600 + \frac{0.20 - 0.600}{4} = 0.600 - 0.100 = \mathbf{0.500} \quad (\text{Identical})$$

5. **Exercise 5:** The parent's, reduced by the FPU term. It cannot be the child's because the child position has not been created or evaluated — there is nothing to read.

6. **Exercise 6:** Almost nothing: 3 samples out of 10,000, about 0.03% of the weight. This is Chapter 9 restated.
7. **Exercise 7:** The *visited policy mass* grew from 0.4513 to 0.8936 when Kf6 was visited, so the reduction $0.33 \cdot \sqrt{\text{mass}}$ grew from 0.22169 to 0.31195. The engine gets more sceptical of untried moves as the explored share rises.

Chapter 11

One-Page Summary

Established mechanism: The Core Principles of Neural MCTS

The Network (once per new position):

Value head $\rightarrow w, d, l \rightarrow V = w - l$ (how the position stands)

Policy head $\rightarrow P$ per move (where to look first)

Every Arrow carries: P (fixed), n (visits), W (total), $Q = W/n$ (running average).

Selection Rule (at every node, every iteration):

$$S(a) = Q(a) + U(a)$$

$$U(a) = c \cdot P(a) \cdot \frac{\sqrt{N}}{1 + n(a)} \rightarrow \text{take } \arg \max_a S(a)$$

Quality Q (depending on the visit count):

$n = 0 \rightarrow$ borrowed from parent, minus the FPU reduction

$n \geq 1 \rightarrow$ real average of values from child positions (discarded, never blended)

Backup Step : $x_n = -V(\text{child position})$ because sides alternate.

Why It Works :

- Bad moves are sampled once and abandoned, so they barely enter the average.
- Good moves are sampled repeatedly, and each repeat pushes one ply deeper.
- Depth is never scheduled — it emerges from the same move winning again.

Notes on Alignment with the Visual Guide. Companion to *guide/neural_mcts_visual_guide_v2.pdf*. Every number here is taken from that guide's own figure sources (*guide/figures/fig_2_*.tex*), so the two documents agree to five decimal places. The constants $c = 1.745$ and $k = 0.33$ reproduce every published score exactly.

One Subtlety Worth Stating. *The FPU baseline is computed from the parent’s **running average** Q , not from its original network value V . The two differ as soon as children start reporting back — using V here gives 0.75433 where the engine gives 0.75015. $LC0$ ’s real exploration constant also grows slowly with visit count rather than staying fixed, which changes none of the mechanisms described here.*